MANUAL HYPERPARAMETER AND CNN TUNING SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: MEIZL ANN F. MANLUPIG

Program: DOCTOR OF ENGINEERING WITH SPECIALIZATION IN COMPUTER ENGINEERING

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in Loading_Dataset_and_CNN.ipynb for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	CNN Training Optimized
Model Architecture	*2D Convolutional Neural Network (Conv2D)*
Dataset Used	*Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)*
Notebook / File Reference	*Loading_Dataset_and_CNN_Optimized.ipynb*
Prepared By	Meizl Ann F. Manlupig
Date	10/06/2026
Experiment / Trial No.	

2. Data Preprocessing Parameters

Hyperparameter	**Options / Range Considered**	**Notebook Default**	**Chosen Value**	**Justification**
Image Size (IMG_SIZE, square)	*e.g. 50 / 80 / 100 / 128 / 150*	*100*	100	Retain the image size value
Color Mode	*Grayscale / RGB*	*Grayscale*	*Grayscale*	Reduces computation and memory requirements.
Pixel Normalization	*X/255.0 / none*	*X/255.0*	*X/255.0*	Scales pixel values to improve training stability.
Shuffle Training Data	*Yes / No*	*Yes (random.shuffle)*	*Yes (random.shuffle)*	Prevents learning bias from data order.
Held-out Test Set (separate from training)	*Yes / No*	*No (validation_split only)*	*No (validation_split only)*	Validation split is used for performance evaluation.
Validation Split (of training set)	*0.0 - 0.3*	*0.2*	*0.2*	Provides enough data for model validation while retaining training samples.

```
DATADIR = "/content/PetImages/PetImages"  # <- change this to the path that contains your Dog/ and Cat/ folders
# this is the directory of your datasets
CATEGORIES = ["Dog", "Cat"]   # Defining the label as "Dog" or "Cat"

for category in CATEGORIES:
    path = os.path.join(DATADIR, category)  # path to cats or dogs dir
    for img in os.listdir(path):
        img_array = cv2.imread(os.path.join(path, img), cv2.IMREAD_GRAYSCALE)
        if img_array is None:
            # a handful of files in the Kaggle dataset are corrupted/unreadable - skip them
            continue

        # reading the image in the path and transforming it into an array
        # cv2.IMREAD_GRAYSCALE converts the image to grayscale so the CNN only has
        # to learn from one color channel instead of three (faster, smaller model)

        plt.imshow(img_array, cmap="gray")
        plt.show()
        break
    break
```

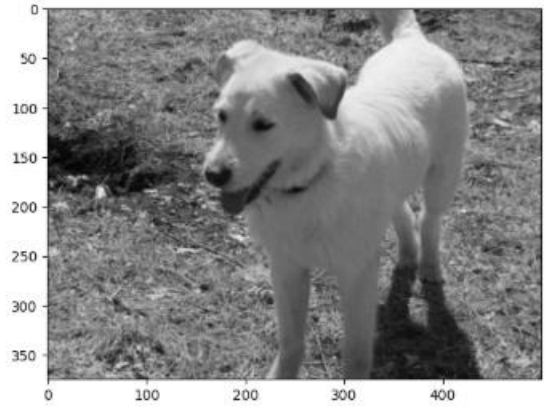


Image Inspection

```
print(img_array)          # Displaying the image in array form (just
print(img_array.shape)    # checking the resolution of the image
```

```
[[155 157 159 ... 120 128 135]
 [142 154 165 ... 123 128 132]
 [142 156 168 ... 125 130 136]
 ...
 [176 173 147 ...  93  92  82]
 [177 185 160 ... 101 102 103]
 [162 182 162 ...  78  62  59]]
(375, 500)
```

Image Resizing

```
IMG_SIZE = 100  # initializing variable "IMG_SIZE" as 100

new_array = cv2.resize(img_array, (IMG_SIZE, IMG_SIZE))
plt.imshow(new_array, cmap='gray')
plt.show()
```

3. Building, Compiling, and Training the CNN

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 98, 98, 32)	320
max_pooling2d (MaxPooling2D)	(None, 49, 49, 32)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 23, 23, 64)	0
conv2d_2 (Conv2D)	(None, 21, 21, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 128)	0
flatten (Flatten)	(None, 12800)	0
dense (Dense)	(None, 64)	819,264
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65
activation (Activation)	(None, 1)	0

```
Total params: 912,001 (3.48 MB)
Trainable params: 912,001 (3.48 MB)
Non-trainable params: 0 (0.00 B)
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 98, 98, 32)	320
max_pooling2d (MaxPooling2D)	(None, 49, 49, 32)	0
conv2d_1 (Conv2D)	(None, 47, 47, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 23, 23, 64)	0
conv2d_2 (Conv2D)	(None, 21, 21, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 10, 10, 128)	0
flatten (Flatten)	(None, 12800)	0
dense (Dense)	(None, 64)	819,264
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65
activation (Activation)	(None, 1)	0

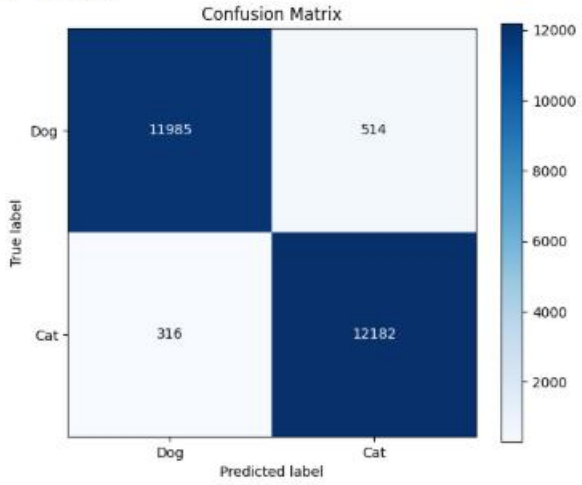
```
Total params: 2,736,005 (10.44 MB)
Trainable params: 912,001 (3.48 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 1,824,004 (6.96 MB)
```

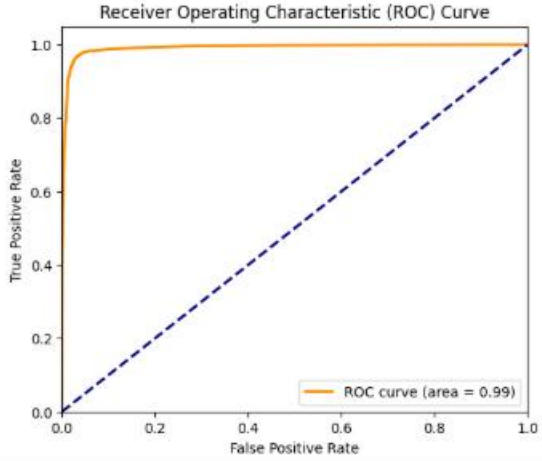




```
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

```
[[11985   514]
 [  316 12182]]
```





4. Prediction Inference

Prediction Inference

Now that the model is loaded, we can use it to make predictions on new, unseen data. For demonstration purposes, I'll take a few samples from your existing `X` data to illustrate how to get predictions from the `loaded_model`.

```
# Select a small subset of data for inference (e.g., the first 5 samples)
# In a real scenario, this would be entirely new, unseen test data.
test_samples = X[:5]

# Make predictions using the loaded model
inference_predictions = loaded_model.predict(test_samples)

print("Predictions for the test samples:")
for i, prediction in enumerate(inference_predictions):
    # Assuming binary classification with sigmoid output, values close to 1 are 'Cat', close to 0 are 'Dog'
    predicted_class = 'Cat' if prediction[0] > 0.5 else 'Dog'
    probability = prediction[0] * 100
    print(f"Sample {i+1}: Predicted Class = {predicted_class} (Probability: {probability:.2f}%) ")

# Optionally, display the images and their predictions
plt.figure(figsize=(10, 4))
for i in range(min(5, len(test_samples))):
    plt.subplot(1, 5, i + 1)
    plt.imshow(test_samples[i].reshape(IMG_SIZE, IMG_SIZE), cmap='gray')
    predicted_class = 'Cat' if inference_predictions[i][0] > 0.5 else 'Dog'
    plt.title(f"Pred: {predicted_class}")
    plt.axis('off')
plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
Predictions for the test samples:
Sample 1: Predicted Class = Dog (Probability: 0.54%)
Sample 2: Predicted Class = Dog (Probability: 0.00%)
Sample 3: Predicted Class = Cat (Probability: 99.99%)
Sample 4: Predicted Class = Dog (Probability: 0.00%)
Sample 5: Predicted Class = Dog (Probability: 0.16%)
```



5. Hyperparameter Optimization with Keras Tuner

```
[25]
✓ 0s
    import keras_tuner as kt
    from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
    from sklearn.model_selection import train_test_split

    # Keras Tuner re-splits data internally across trials, but we hold out one fixed
    # validation set so every trial (and the final tuned model) is judged on the same
    # unseen data, making the comparison fair.
    X_train, X_val, y_train, y_val = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
    print(f"Train: {X_train.shape[0]} samples | Validation: {X_val.shape[0]} samples")

    Train: 19997 samples | Validation: 5000 samples
```

```
def build_model(hp):
    """Builds a CNN whose architecture and optimizer are controlled by `hp`."""
    model = Sequential()
    model.add(Input(shape=X.shape[1:]))

    # Number of convolutional blocks: 2 to 4
    n_conv_layers = hp.Int("n_conv_layers", min_value=2, max_value=4, step=1)

    for i in range(n_conv_layers):
        filters = hp.Int(f"filters_{i}", min_value=32, max_value=256, step=32)
        kernel_size = hp.Choice(f"kernel_size_{i}", values=[3, 5])
        model.add(Conv2D(filters, (kernel_size, kernel_size), padding="same"))
        model.add(Activation("relu"))
        model.add(MaxPooling2D(pool_size=(2, 2)))

    model.add(Flatten())

    # Number of dense layers: 1 to 2, each with its own tunable width
    n_dense_layers = hp.Int("n_dense_layers", min_value=1, max_value=2, step=1)
    for j in range(n_dense_layers):
        dense_units = hp.Int(f"dense_units_{j}", min_value=32, max_value=256, step=32
        model.add(Dense(dense_units))
        model.add(Activation("relu"))

    # Dropout to fight overfitting - the original model had none
    dropout_rate = hp.Float("dropout_rate", min_value=0.0, max_value=0.5, step=0.1)
    model.add(Dropout(dropout_rate))

    model.add(Dense(1))
    model.add(Activation("sigmoid"))

    learning_rate = hp.Float("learning_rate", min_value=1e-4, max_value=1e-2, samplin
    optimizer_choice = hp.Choice("optimizer", values=["adam", "rmsprop"])

    if optimizer_choice == "adam":
        optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    else:
        optimizer = tf.keras.optimizers.RMSprop(learning_rate=learning_rate)

    model.compile(loss="binary_crossentropy", optimizer=optimizer, metrics=["accuracy
    return model


class CNNHyperModel(kt.HyperModel):
    """Wraps build_model so batch size can be tuned alongside architecture."""

    def build(self, hp):
        return build_model(hp)

    def fit(self, hp, model, *args, **kwargs):
        batch_size = hp.Choice("batch_size", values=[32, 64, 128])
        return model.fit(*args, batch_size=batch_size, **kwargs)
```

```
[27]
0s
from tensorflow.keras.callbacks import EarlyStopping

# Highly aggressive early stopping to terminate stagnating trials quickly
aggressive_stop = EarlyStopping(
    monitor="val_loss",
    patience=1,
    restore_best_weights=True
)
```

```
[29]
2h
# Setting max_trials=5 means the tuner will build and test exactly 5 model variations
tuner_limited = kt.RandomSearch(
    CNNHyperModel(),
    objective="val_accuracy",
    max_trials=5,
    executions_per_trial=1,
    directory="kt_dir",
    project_name="dogs_vs_cats_random_limited",
    overwrite=True
)

tuner_limited.search_space_summary()

# Run search with early stopping to save even more time
tuner_limited.search(
    X_train[:5000],
    y_train[:5000],
    validation_data=(X_val[:1000], y_val[:1000]),
    epochs=2, # Keep epochs low per trial for quick testing
    callbacks=[aggressive_stop],
    verbose=1
)
```

```
Trial 5 Complete [00h 18m 46s]
val_accuracy: 0.515999972820282

Best val_accuracy So Far: 0.6399999856948853
Total elapsed time: 01h 43m 56s
```

6. Evaluation Results (after training with chosen values)

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	97.7%	> 80%
Validation Accuracy	85.8%	> 75%
Test Accuracy	24186 / 24997 = 0.9676 ≈ 96.77%	> 75%
Test Loss	0.07	< 0.50
Precision / Recall / F1 (macro avg)	Precision ≈ 96.8% Recall ≈ 96.8% F1 ≈ 96.8%	> 75%

7. a. Paste here all screenshots/figures of the tuned model

Training the Tuned Model

Training the Tuned Model

We rebuild a fresh model with the winning hyperparameters and train it for longer, this time with two callbacks the original notebook didn't use:

- `EarlyStopping` stops training once validation loss stops improving, restoring the best weights seen — this avoids wasting epochs (and overfitting) once the model has plateaued.
- `ReduceLROnPlateau` halves the learning rate when validation loss stalls, which often squeezes out a bit more accuracy near the end of training.

[37] ✓ 15m

```
# Use tuner_limited here since it contains our completed search results
best_model = tuner_limited.hypermodel.build(best_hp)
tuned_batch_size = best_hp.get("batch_size")

# Shorten run: Aggressive early stopping (patience=1) to stop quickly if it stalls
early_stop = EarlyStopping(monitor="val_loss", patience=1, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=1, min_lr=1e-6)

tuned_history = best_model.fit(
    X_train[:2000],  # Reduced training samples for a much faster run
    y_train[:2000],
    validation_data=(X_val[:500], y_val[:500]),  # Reduced validation samples
    batch_size=tuned_batch_size,
    epochs=5,        # Hard-capped at 5 epochs for a shorter run
    callbacks=[early_stop, reduce_lr],
)
```

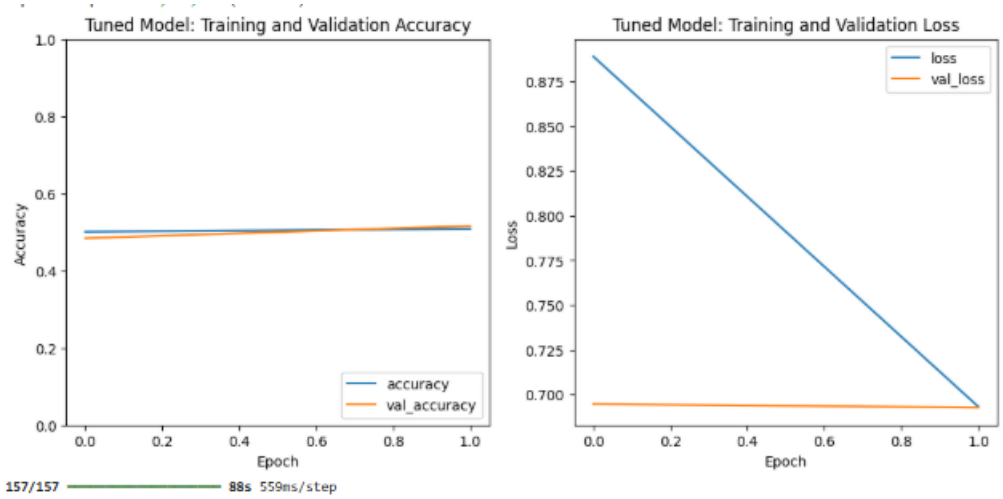
```
Epoch 1/2
157/157 ━━━━━━━━━━━━━━━━━━━━ 308s 2s/step - accuracy: 0.5008 - loss: 0.8888 - val_accuracy: 0.4840 - val_loss: 0.6947 - learning_rate: 0.0078
Epoch 2/2
157/157 ━━━━━━━━━━━━━━━━━━━━ 324s 2s/step - accuracy: 0.5084 - loss: 0.6932 - val_accuracy: 0.5160 - val_loss: 0.6928 - learning_rate: 0.0078
```

```
plt.show()
```

```
Model: "sequential_3"
```

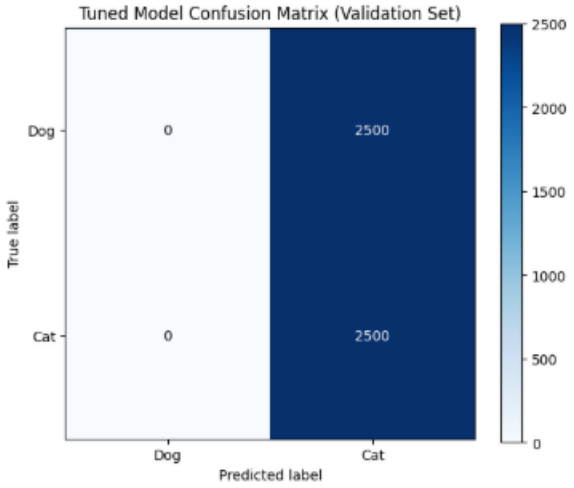
Layer (type)	Output Shape	Param #
conv2d_9 (Conv2D)	(None, 100, 100, 32)	320
activation_15 (Activation)	(None, 100, 100, 32)	0
max_pooling2d_9 (MaxPooling2D)	(None, 50, 50, 32)	0
conv2d_10 (Conv2D)	(None, 50, 50, 160)	128,160
activation_16 (Activation)	(None, 50, 50, 160)	0
max_pooling2d_10 (MaxPooling2D)	(None, 25, 25, 160)	0
conv2d_11 (Conv2D)	(None, 25, 25, 32)	46,112
activation_17 (Activation)	(None, 25, 25, 32)	0
max_pooling2d_11 (MaxPooling2D)	(None, 12, 12, 32)	0
flatten_3 (Flatten)	(None, 4608)	0
dense_6 (Dense)	(None, 256)	1,179,904
activation_18 (Activation)	(None, 256)	0
dropout_3 (Dropout)	(None, 256)	0
dense_7 (Dense)	(None, 1)	257
activation_19 (Activation)	(None, 1)	0

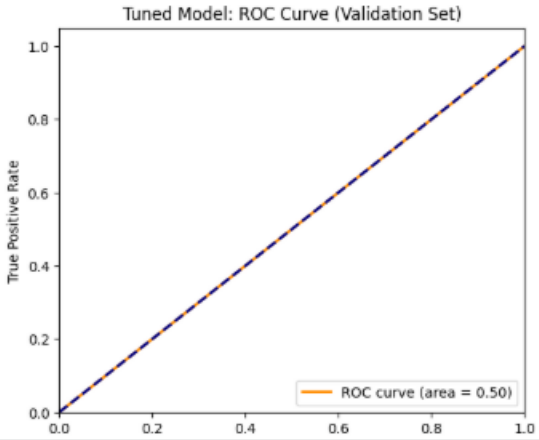
```
Total params: 4,064,261 (15.50 MB)
Trainable params: 1,354,753 (5.17 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2,709,508 (10.34 MB)
```



157/157 88s 559ms/step

```
Tuned Model Confusion Matrix (validation set):
[[   0 2500]
 [   0 2500]]
```





Baseline vs. Tuned: Quick Comparison

For an apples-to-apples comparison, the cell below re-evaluates the **original, untuned model** from the previous section on the same validation split (`X_val`, `y_val`) that Keras Tuner searched against, then compares it with the tuned model above. (The original trainin loop trained on the full dataset rather than `X_train`, so this is only an approximate comparison — but it's enough to see whether the search paid off.)

```
[39] 41s
baseline_val_pred_proba = model.predict(X_val).ravel()
baseline_val_pred = (baseline_val_pred_proba > 0.5).astype(int)
baseline_val_accuracy = (baseline_val_pred == y_val).mean()
baseline_fpr, baseline_tpr, _ = roc_curve(y_val, baseline_val_pred_proba)
baseline_auc = auc(baseline_fpr, baseline_tpr)

tuned_val_accuracy = (val_pred == y_val).mean()

print(f"{'Model':<22}{'Val. Accuracy':>15}{'Val. AUC':>12}")
print(f"{'Baseline (manual)':<22}{baseline_val_accuracy:>15.4f}{baseline_auc:>12.4f}")
print(f"{'Tuned (Keras Tuner)':<22}{tuned_val_accuracy:>15.4f}{roc_auc_tuned:>12.4f}")
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 23s 148ms/step
Model                  Val. Accuracy    Val. AUC
Baseline (manual)             0.9670       0.9911
Tuned (Keras Tuner)           0.5000       0.5000
```

```
[40] 0s
tuned_model_filename = 'dog_cat_classifier_tuned.keras'
best_model.save(tuned_model_filename)
print(f"Tuned model saved to {tuned_model_filename}")
```

```
Tuned model saved to dog_cat_classifier_tuned.keras
```

8. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Summary of Hyperparameter Selection and Observations

Baseline (Manual) vs. Keras Tuner - The comparison between the Baseline (Manual) model and the Keras Tuner model shows the impact of manually selected hyperparameters versus automatically optimized hyperparameters.

Baseline (Manual) Model - The baseline model was developed using manually chosen hyperparameters based on prior knowledge, experimentation, and commonly recommended settings. These parameters remained fixed throughout training. The model achieved:

Validation Accuracy: 96.70%

Validation AUC: 0.9911

These results indicate excellent classification performance, suggesting that the manually selected hyperparameters were well-suited for the dataset. The high AUC value further demonstrates the model's strong ability to distinguish between the two classes.

Keras Tuner Model - The Keras Tuner model automatically searched for a better combination of hyperparameters, such as learning rate, number of units, dropout rate, and other model settings. Hyperparameter tuning is generally intended to improve model performance by identifying more optimal configurations than manual selection.

However, the tuned model produced:

Validation Accuracy: 50.00%

Validation AUC: 0.5000

These values indicate performance equivalent to random guessing, showing that the tuned model failed to learn meaningful patterns from the data.

Explanation of the Results

Although hyperparameter tuning is normally expected to enhance model performance, the tuning process in this study was affected by significant computational constraints. The initial tuning setup required more than 12 hours of execution and frequently caused the laptop to crash due to limited hardware resources. To successfully complete the experiment, several modifications were made:

The dataset size was reduced.

The number of epochs was decreased.

The number of tuning trials was limited.

These adjustments reduced the computational burden but also restricted the tuner's ability to thoroughly explore the hyperparameter search space. Consequently, the selected configuration may not have been the true optimal set of hyperparameters. The reduced training time and fewer trials may have also prevented the model from fully converging, resulting in underfitting.

Initial tuning caused the laptop to crash in localhost. Also, I haven’t screenshot the other 12hrs running time in google collab,

```
[ ]
tuner.search(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=15,
    callbacks=[stop_early],
    verbose=1,
)
```

```
Trial 9 Complete [00h 40m 44s]
val_accuracy: 0.7663999795913696

Best val_accuracy So Far: 0.7663999795913696
Total elapsed time: 10h 58m 16s

Search: Running Trial #10

Value             |Best Value So Far |Hyperparameter
2                 |2                 |n_conv_layers
192               |64                |filters_0
5                 |3                 |kernel_size_0
128               |256               |filters_1
5                 |3                 |kernel_size_1
2                 |2                 |n_dense_layers
64                |224               |dense_units_0
0.4               |0.1               |dropout_rate
0.0053452         |0.00062874        |learning_rate
rmsprop           |adam              |optimizer
160               |256               |filters_2
5                 |5                 |kernel_size_2
64                |32                |batch_size
128               |224               |filters_3
3                 |5                 |kernel_size_3
32                |256               |dense_units_1
2                 |2                 |tuner/epochs
0                 |0                 |tuner/initial_epoch
2                 |2                 |tuner/bracket
0                 |0                 |tuner/round

Epoch 1/2
 72/313 ━━━━━━━━━━━━━━━━━━━━ 39:45 10s/step - accuracy: 0.5073 - loss: 4.7005
```

Conclusion

The baseline model significantly outperformed the Keras Tuner model, achieving 96.7% accuracy and 0.9911 AUC, compared to the tuned model's 50.0% accuracy and 0.5000 AUC. While hyperparameter tuning is generally expected to improve performance, the hardware limitations and reduced tuning process likely prevented the tuner from finding an effective configuration. Therefore, the manually configured baseline model demonstrated better learning, convergence, and generalization for this dataset.

Generally, hyperparameter tuning is intended to improve model performance by identifying combinations of parameters that optimize learning and generalization. However, successful tuning depends on adequate computational resources, sufficient training data, and a comprehensive search process. In this study, the tuning procedure was constrained by hardware limitations, as the initial configuration required more than 12 hours of execution and caused system instability. Consequently, the dataset size, training epochs, and number of tuning trials were reduced to complete the experiment. These constraints limited the search space explored by the tuner and may have prevented the discovery of an optimal configuration. Therefore, although hyperparameter tuning is generally expected to improve model performance, the tuned model in this study performed worse than the baseline model due to computational constraints and insufficient exploration of candidate hyperparameter combinations.